

Self-Organizing Map Oversampling (SOMO) for imbalanced data set learning



Georgios Douzas, Fernando Bacao*

NOVA IMS, Universidade Nova de Lisboa, 1070-312 Lisboa, Portugal

ARTICLE INFO

Article history:

Received 1 November 2016

Revised 14 February 2017

Accepted 30 March 2017

Available online 31 March 2017

ABSTRACT

Learning from imbalanced datasets is challenging for standard algorithms, as they are designed to work with balanced class distributions. Although there are different strategies to tackle this problem, methods that address the problem through the generation of artificial data constitute a more general approach compared to algorithmic modifications. Specifically, they generate artificial data that can be used by any algorithm, not constraining the options of the user. In this paper, we present a new oversampling method, Self-Organizing Map-based Oversampling (SOMO), which through the application of a Self-Organizing Map produces a two dimensional representation of the input space, allowing for an effective generation of artificial data points. SOMO comprises three major stages: Initially a Self-Organizing Map produces a two-dimensional representation of the original, usually high-dimensional, space. Next it generates within-cluster synthetic samples and finally it generates between cluster synthetic samples. Additionally we present empirical results that show the improvement in the performance of algorithms, when artificial data generated by SOMO are used, and also show that our method outperforms various oversampling methods.

© 2017 Elsevier Ltd. All rights reserved.

1. Introduction

An imbalanced learning problem can be defined as a learning problem from a binary or multiple-class data set where the distribution of instances among classes differs significantly (Chawla, Bowyer, Hall, & Kegelmeyer, 2002). The class having the most of the observations is called the majority class and the rest are called the minority classes. Imbalance Ratio (IR) is defined as the ratio between the majority class and each of the minority classes. This ratio is problem dependent and for binary problems values between 100 and 100,000 have been observed (Chawla et al., 2002), (Barua, Islam, Yao, & Murase, 2014). Learning from imbalanced data is an important problem for the research community and industry practitioners. Imbalanced data is “pervasive and ubiquitous” being a persistent and complex hurdle for many practitioners (Chawla, Japkowicz, & Kolcz, 2003). The recent growth of interest by researchers on this subject is an indication of their importance (Chawla, Japkowicz, & Kolcz, 2003; Chawla, Japkowicz, & Drive, 2004; Japkowicz, 2000).

Dealing with this type of data and to still be able to make precise predictions is a significant problem for diverse categories

of machine learning activities and real-world applications such as medical diagnosis, information retrieval systems, fraud detection, detection of oil spills in radar images, direct marketing, automatic classification of land use and land cover in remote sensing images, detection of rare particles in experimental high-energy physics, telecommunications management and bioinformatics (Akbani, Kwek, & Japkowicz, 2004; He & Garcia, 2009; Clearwater & Stern, 1991; Graves et al., 2016; Verbeke, Dejaeger, Martens, Hur, & Baesens, 2012; Zhao, Li, Chen, & Aihara, 2008). It is important to note that in most of these application the data imbalance is inherent to the problem (e.g. there are much fewer cases of fraud than honest transactions) (Chawla et al., 2004; He & Garcia, 2009). Thus, in most cases the ability to develop good models is entirely dependent on the ability to learn from (highly) imbalanced data. Additionally, the misclassification cost of the minority class is often higher than the misclassification cost of the majority class (Domingos, 1999; Ting, 2002). For instance, the misclassification cost of a fraud transaction as a normal one has a much higher cost for the organization than the inverse scenario. Another example is the identification of rare or even unknown particles in experimental high-energy physics (Clearwater & Stern, 1991). These positive examples represent only a small fraction of the data produced from the collisions of particles in the accelerators. In this case, low accuracy of the classifier for the detection of the rare particles means that the experiment fails to identify new interest-

* Corresponding author.

E-mail addresses: gdouzas@icloud.com (G. Douzas), bacao@novaims.unl.pt (F. Bacao).

ing phenomena. The inverse case of incorrectly identifying known particles as rare does not have a high cost and can be ruled out based on theoretical arguments.

Standard learning methods perform poorly in imbalanced data sets as they assume balanced class distributions or equal misclassification costs. As explained above, in real world problems, the minority classes are the ones where the classifier is preferred to achieve the highest accuracy (Prati, Batista, & Silva, 2014). The application of standard algorithms induces a bias in favor of the majority class for two main reasons. First, during the training phase of any classifier the minority classes contribute less to the minimization of the classifier's objective function and second, the distinction between noise instances and minority class instances is often difficult. Also the fact that the misclassification cost of the minority classes could be higher than the misclassification cost of the majority class is not addressed.

The machine learning community has approached the issue of class imbalance in three ways. One is the modification at the data level by rebalancing the class distribution using under-sampling, over-sampling or hybrid methods. The second is the modification or creation of algorithms that reinforce the learning towards the minority class. The last approach is the application of cost-sensitive methods at the data or algorithmic level in order to minimize higher cost errors (Fernández, López, Galar, Jesus, & Herrera, 2013). Methods that address the problem by modification on the data level and particularly oversampling methods that address the problem through the generation of artificial data, constitute a more general approach compared to algorithmic modifications. Specifically, they generate artificial data that can be used during the learning phase by any algorithm.

In this paper a new oversampling method, called Self-Organizing Map-based Oversampling (SOMO), is presented for imbalanced binary classification problems. Initially the Self Organizing Map algorithm (Kohonen, 2001) is applied to the data set. The SOM describes a mapping from the, usually higher dimensional, input space to a two-dimensional map space. The resulting map partitions the data into clusters and produces a two-dimensional discretized representation of the input space, which has the property of preserving the original topology (Merenyi, Tasdemir, & Zhang, 2009). A subset of these clusters is chosen and a weight is assigned to them based on the density of the minority class examples. Then synthetic examples are generated by exploiting the preservation of the topological properties of the input space where their distribution depends on the calculated intracluster and intercluster weights.

For the evaluation of SOMO an experimental analysis is performed, based on thirteen publicly available datasets from Machine Learning Repository as well as thirteen modified versions of these datasets, which are classified using two different classifiers. The results of SOMO are then compared to Random Oversampling, Synthetic Minority Oversampling Technique (SMOTE) algorithm (Chawla et al., 2002), Borderline SMOTE (Han, Wang, & Mao, 2005), Cluster SMOTE (Cieslak, Chawla, & Striegel, 2006) and ADASYN (He, Bai, Garcia, & Li, 2008).

The sections in the paper are organized as follows. In Section 2, an overview of related previous works and existing sampling methods is given. In Section 3, the motivation behind SOMO algorithm is described. Section 4 presents the proposed method in detail. Section 5 presents the research methodology. In Section 6 the experimental results are presented while conclusions are provided in Section 7.

2. Related work

Considering that our focus is the modification on the data level, known as sampling methods, we provide a short review of

these approaches. A review of the other methods can be found in (Galar, Fernández, Barrenechea, Bustince, & Herrera, 2012), (Chawla, 2010). Generally sampling methods can be categorized as undersampling and oversampling. Undersampling reduces the number of majority class samples by removing samples from the training set. On the other hand oversampling generates synthetic examples for the minority class and adds them to the training set. These methods can be further categorized into random and heuristic approaches. The first selects random examples and the latter takes into account the distribution of the instances of the given imbalance learning problem. It has been shown that both oversampling and undersampling can be effective and their effectiveness depends on the specific problem that is addressed (Chawla et al., 2002). Comparing the two methods we may observe that undersampling excludes information from the learning process which may negatively affect the performance of the classifier in cases where the data set is small (He et al., 2008). On the other hand oversampling creates extra information by generating synthetic examples. Also, informed oversampling compared to random oversampling avoids the problem of producing artificial data that can lead to overfitting (He & Garcia, 2009).

Random undersampling is a non-heuristic method that improves the balance of the classes by randomly excluding some of the majority class examples. This approach has the disadvantage that since no selection criteria are applied for these examples, potentially useful information for the learning process may be excluded from the training phase. To deal with this problem various informative undersampling methods have been proposed such as the Neighborhood Cleaning Rule (Laurikkala, 2001), Tomek Links (Tomek, 1976), the Condensed Nearest Neighbor (CNN) rule (Hart, 1968) and One-Sided Selection (Kubat & Matwin, 1997). The Neighborhood Cleaning Rule aims to eliminate information that can be considered as noisy by removing either an observation i which belongs to the majority class and it is misclassified by its three nearest neighbors, or its three nearest neighbors when i belongs to the minority class. The Tomek Links approach first identifies pairs of minority/majority class samples, based on the absence of a third observation that has a Euclidean distance to both of them greater than the Euclidean distance of the pair, and then removes the majority class sample. CNN removes instances from the majority class that are not close to the decision boundary. The remaining examples are considered as safe and are used for the learning process. Finally One-Sided Selection is an undersampling approach that identifies Tomek Link pairs and also applies the Condensed Nearest Neighbor (CNN) rule.

Similarly to random undersampling, random oversampling is a non-informed method that improves the balance of the classes through the random replication of minority class instances. The drawback of this approach is that the exact replication of training examples can lead to overfitting, when a supervised algorithm is applied to the training set, since information is repeated. A more informative approach that avoids this problem is SMOTE. The minority class is oversampled by taking a minority class observation and introducing synthetic examples along the line segment which joins any of the k minority class nearest neighbors with it. SMOTE has the disadvantage that, since majority and minority class clusters may not be clearly separated in some areas of the input space, the generation of artificial data may increase the overlap between them (He & Garcia, 2009). To avoid this scenario various approaches have been proposed. SMOTE+Edited Nearest Neighbor (Batista, Prati, & Monard, 2004) combination applies the edited nearest neighbor rule (Wilson, 1972) after the generation of artificial examples through SMOTE to remove any misclassified instances, based on the classification by its three nearest neighbors. Another approach is Safe-Level SMOTE (Bunkhumpornpat, Sinapiromsaran, & Lursinsap, 2009). Safe-Level

SMOTE modifies the SMOTE algorithm by applying a weight degree, the safe level, in the data generation process. The safe level provides a scale to differentiate between noisy and safe instances. Borderline-SMOTE (Han et al., 2005), MWMOTE (Majority Weighted Minority Oversampling Technique for Imbalanced Data Set Learning) (Barua et al., 2014), ADASYN (He et al., 2008) and its variation KernelADASYN (Tang & He, 2015) aim to avoid noisy samples for the generating process and to choose hard-to-learn instances. This is achieved by identifying the borderline instances of the majority and minority classes that in turn are used to identify the informative minority class samples. The methods described above deal with the problem of between-class imbalance (Nekooimehr & Lai-Yuen, 2016). Different types of methods that deal with the within-class imbalance problem (Nekooimehr & Lai-Yuen, 2016; Bunkhumpornpat, Sinapiromsaran, & Lursinsap, 2012; Cieslak & Chawla, 2008; Jo & Japkowicz, 2004) i.e. when sparse or dense subclusters of minority or majority instances exist, have recently been presented. These are clustering based methods that first partition the input space and then sampling methods are applied to adjust the size of the various clusters. Cluster-SMOTE (Cieslak et al., 2006) initially applies the k-means algorithm and then SMOTE is used in the clusters in order to generate artificial data. Similarly DBSMOTE (Bunkhumpornpat et al., 2012) uses the DB-SCAN algorithm to discover arbitrarily shaped clusters and then generates synthetic instances along a shortest path from each minority class instance to a pseudo-centroid of the cluster. A more sophisticated approach that belongs in the same category is A-SUWO (Nekooimehr & Lai-Yuen, 2016). A-SUWO creates clusters of the minority class instances with a size, which is determined using cross validation and generates synthetic instances based on a proposed weighting system. Other approaches are based on ensemble methods such as SMOTEBoost (Chawla, Lazarevic, Hall, & Bowyer, 2003) and DataBoost-IM (Guo & Viktor, 2004). Finally, Wang, Minku, and Yao (2015) and Sun, Song, Zhu, Sun, Xu, and Zhou (2015) are more recent references in the same category of ensemble based methods.

SOMO, the algorithm we propose, addresses the problems of both between-class imbalance and within-class imbalance. SOMO can be categorized as a clustering based oversampling approach that takes into account the topological characteristics of complex high dimensional spaces.

3. Motivation

The previous section mentioned that informative oversampling methods have been shown to be effective in dealing with imbalance data. However, there are cases where existing methods may encounter a variety of problems. In this section we describe some

of these scenarios and motivate the choice of SOM as a clustering and dimensionality reduction algorithm applied to the input space. Some of the inefficiencies of the current oversampling methods are the following:

1. The generation of noisy examples that penetrate in the area of the majority class examples.

SMOTE, as well as many other methods, applies the k -nearest neighbor approach during the sample generation phase. In order to generate an artificial example g from an existing minority class example x , first the k NN approach randomly selects another minority class sample y from the k -nearest neighbors of x . Then g is generated by using a linear interpolation of x and y which can be expressed as

$$g = x + a \cdot (y - x)$$

where a is a random number in the range $[0, 1]$. Thus g lies in the line segment between x and y . Nevertheless the appropriate value of k cannot be determined in advance and the results of oversampling are sensitive to it, as is shown in the next examples. A large value can result in the generation of noisy samples since y , the k -nearest neighbor of x , might be a noisy sample that has penetrated in the majority class area as is shown in Fig. 1(a). Even a small value of k does not always avoid the above scenario since x might already be a noisy sample. This scenario can also be seen in Fig. 1(b). The generation of noisy samples can reduce the accuracy of predictions for both the minority and majority classes (Kotsiantis, Kanellopoulos, & Panayiotis, 2006).

2. The generation of nearly duplicated samples.

As was mentioned above a small value of k can be chosen in order to avoid the generation of noisy samples. Another approach is to generate a new instance between a selected observation and one of its neighbors from the same cluster (Barua et al., 2014). Both of these choices have the consequence that the generation of synthetic examples might occur in dense minority class areas as is shown in Fig. 2(a). These samples are less useful because they do not add any new information to the imbalanced data set and may lead to overfitting. Thus it is desirable to expand the data generation process in areas where minority examples are absent. Fig. 2(b) presents a scenario where the number of k -nearest neighbors is increased in order to generate two intercluster instances. This attempt results in the generation of a noisy instance.

3. Use of heuristics that are based on the assumption that the input space has a simple manifold structure.

To deal with the aforementioned problems modifications of SMOTE have been proposed. A summary of these approaches

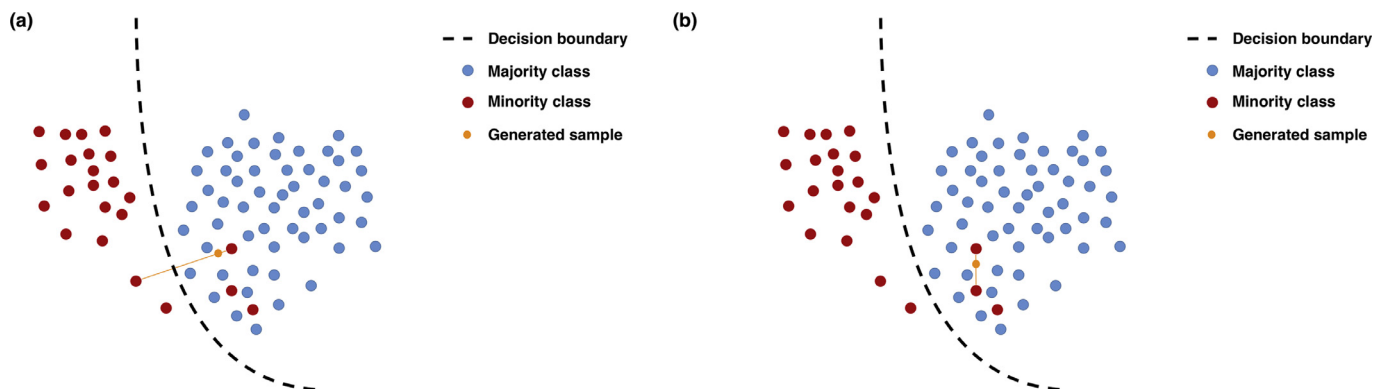


Fig. 1. Two cases where current methods are insufficient. New instances (orange dots) are generated in the line segment between two minority class instances (red dots). Majority class instances (blue dots) are also shown. (a) An instance near the decision boundary and one of its 4-nearest neighbors are selected randomly. A noisy observation is generated. (b) A 2-nearest neighbor does not avoid the generation of noise when noisy samples are initially selected.

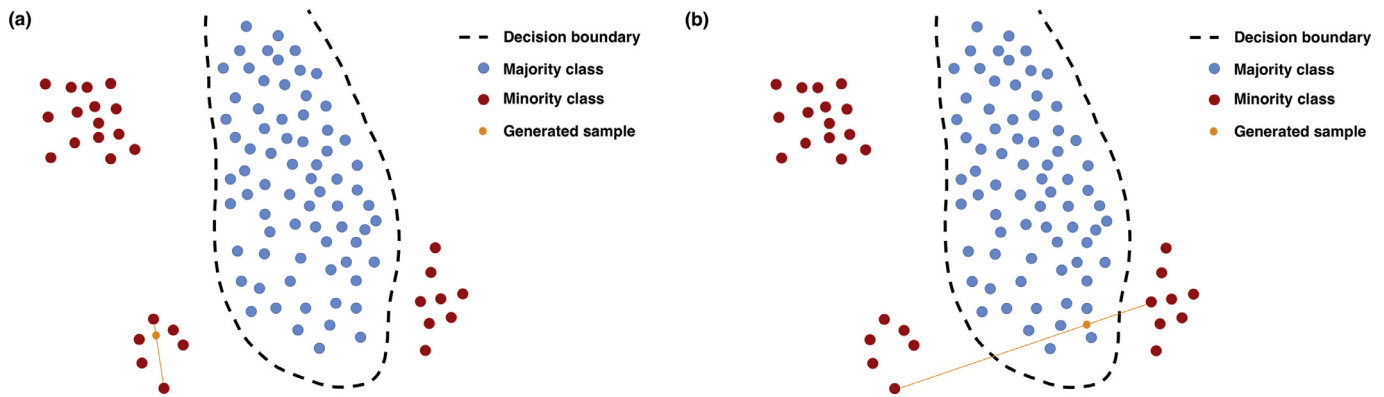


Fig. 2. (a) An instance belonging to a minority class cluster and one of its 5-nearest neighbors are selected. An observation belonging to the same cluster is generated. (b) An attempt to generate intercluster instances by increasing the number of k -nearest neighbors of the selected instances. One of the generated instances penetrates in the majority class area.

was presented in the previous section. A common characteristic they share is that they aim to avoid noisy samples in the generating process and to choose hard-to-learn instances. These approaches are based on the parameterization of the data generation process based on heuristic methods that apply in two-dimensional spaces. On the other hand, practical applications very often include high dimensional data that are represented by highly structured manifolds. These manifolds include clusters which can have very different characteristics such as size and shape as well as statistical properties (Merenyi, Tasdemir, & Zhang, 2009). Additionally, because of the “Curse of Dimensionality” effect (Bellman, 1961; Aggarwal, Hinneburg, & Keim, 2001) the data points become sparse as the dimensionality increases and under some reasonable assumptions on the distribution of the data, the nearest neighbor problem becomes ill defined (Beyer, Goldstein, Ramakrishnan, & Shaft, 1999). Therefore the use of heuristics that are based on the Euclidean distance between points and the assumption of well-defined clusters may fail to apply in high dimensional spaces and result in the generation of noisy instances.

4. The proposed method – SOMO

In the previous section some insufficiencies of the existing methods that may apply in various cases including high dimensional data were described. We propose a novel algorithm, SOMO, which is an informed oversampling method and has two main objectives:

1. To improve the selection criteria of the minority class samples that are used to generate synthetic examples so that the generation of noisy examples is avoided.
2. To improve the distribution of the generated synthetic examples in more productive areas of the data space.

The proposed algorithm is composed of three stages. In the first stage a SOM algorithm is applied to the initial data set and the data points are partitioned into clusters. The SOM usually consists of a two-dimensional grid of map units. Each unit is represented by a codebook vector and the SOM training can be considered as a “rubber surface” that is stretched and bent all over the input space” (Kohonen, 2001) in order to adjust to the data cloud in the input space. Therefore the SOM is a map from the input space to the discretized two-dimensional space of map units. Additionally, during training, data points that are close to each other in the input space are mapped onto nearby map units and thus a neighborhood structure is created that connects the map units to adjacent ones. This topology preserving property of the map may be

exploited to partition the input space into clusters and to identify adjacent clusters that respect the higher dimensional structure of the input space. Thus in the second stage of SOMO synthetic data points for the minority class are generated, using instances of the minority class that belong to the same cluster. Finally in the third stage synthetic data points for the minority class are generated, using instances of the minority class that belong to adjacent clusters in the two-dimensional grid. The distribution of intracluster and intercluster generated data is adjusted based on the cluster density. These heuristics are applied on the two dimensional representation of the input space that is created by the SOM, therefore the arguments for the insufficiency of methods that assume a simple manifold structure of the input space do not apply in this case.

4.1. SOM algorithm

For many years, artificial neural networks have been studied and used to construct information processing systems, inspired by natural biological neural structures. Among the various existing neural network architectures and learning algorithms, Kohonen’s self-organizing map (SOM) is one of most popular neural network models. Teuvo Kohonen proposed the SOM in the beginning of the 1980s (Kohonen, 1982) as a result of his work on associative memory and vector quantization.

One of SOM’s objectives is to extract and visualize the fundamental structures in a dataset, through a map, which results from an unsupervised learning process (Kaski & Kohonen, 1998; Kaski & Lagus, 1996). Because of its ability to preserve the topological relations, the SOM is usually used as a tool for mapping high-dimensional data into one, two, or three-dimensional feature maps. The main advantage of the SOM is that it allows for insight on the underlying structure of the data, through the analysis of the resulting map.

The topological ordering extracts the latent structure of the input space, which is applicable to many purposes (e.g. dimensionality reduction, data visualization, clustering and classification). Various extensions of the SOM have been devised to extend its application to a wide range of applications. In particular, the SOM has been linked with the principal curve and surface and the recently proposed visualization induced SOM (ViSOM) has been shown to represent a discrete principal curve/surface (Gorban, Kegl, Wunsch, & Zinovyen, 2008).

A detailed explanation of the SOM algorithm can be found in Kohonen (1982). The basic idea of a SOM is to map the data patterns onto an n -dimensional grid of units, also called neurons. The grid is known as the output space as opposed to the input space which is the original space of the data patterns. The mapping pro-

cess tries to preserve topological relations in the data, i.e. patterns that are close in the input space will be mapped to units that are close in the output space and vice versa. The output space is usually two-dimensional and most of the implementations of SOM use a rectangular or hexagonal grid of units. Each unit, being an input layer unit, has as many weights as the input patterns and can thus be regarded as a vector in the same space of the patterns. When training a SOM with a given input pattern, the distance between that pattern and every unit in the network is calculated. The algorithm selects the unit that is closest to the input pattern as the winning unit, also known as the best matching unit, and that pattern is mapped onto that unit. Then patterns that are close in the input space will be mapped to neurons that are close in the output space. Thus SOM is “topology preserving” in the sense that neighborhoods are preserved in the mapping process.

For our purposes the SOM can be a very useful tool, as it allows for extracting information about the manifold structure of the input space and identifying a large variety of clusters. Additionally, the reasoning for using SOM as a tool to generate data stems from the fact that it can also be seen as a density estimation tool (Kohonen, 1982; Sarajedini & Chau, 1996; Meřenyi, Jain, & Villmann, 2007). The properties of the SOM as a density estimation tool have not yet been completely established, except for some very particular cases, e.g. the one dimensional case (Ritter & Schulden, 1988; Cottrell, Fort, & Pagès, 1998). In these cases it has been found that the SOM has a bias towards low density areas known as the magnification effect. In spite of the magnification effect, the more input patterns exist in a specific area of the input space the more units are assigned to represent that specific area. Additionally, units that represent that area are neighbors in the output space, consequently they can be labeled with the same label.

4.2. SOMO algorithm

The complete algorithm in pseudo-code is narrated as:

Algorithm

SOMO(S_{maj} , S_{min} , N_{intra} , N_{inter} , $SOM_{parameters}$)

Input

S_{maj} : Set of majority class samples.

S_{min} : Set of minority class samples.

N_{intra} : The total intracluster number of synthetic samples to be generated.

N_{inter} : The total intracluster number of synthetic samples to be generated.

$SOM_{parameters}$: The SOM algorithm parameters.

Procedure begin

1. Train a SOM on the set $S = S_{min} \cup S_{maj}$ with parameters $SOM_{parameters}$.
2. Identify each map unit as a cluster of the set S . Define the set of cluster labels set $cl = \{1, 2, \dots, N_{Grid}^2\}$, where N_{Grid} is the dimension of the SOM grid that belongs to the $SOM_{parameters}$.
3. For each cluster label $i \in cl$ define the number of minority (positive) class examples as $n_{+,i}$ and the number of majority (negative) class examples as $n_{-,i}$.
4. For each cluster label $i \in cl$ calculate the imbalance ratio IR_i in each cluster as $IR_i = \frac{n_{-,i}+1}{n_{+,i}+1}$.
5. Construct the filtered set cl_f of cluster labels as $cl_f = \{i \in cl : IR_i < 1\}$.
6. In each filtered cluster $i \in cl_f$ calculate the average Euclidean distance $dist_i$ across all pairs of positive class instances belonging to the cluster.
7. For each filtered cluster $i \in cl_f$ calculate the density factor as $den_i = \frac{n_{+,i}}{dist_i^2}$.
8. Calculate the density factor of topologically neighboring mapping units i and j as $den_{ij} = den_i + den_j$ for each combination of filtered clusters $i, j \in cl_f$.
9. Generate N_{intra} intracluster synthetic samples for each filtered cluster $i \in cl_f$:
 - (a) Calculate the intracluster weights as $w_{intra,i} = \frac{1/den_i}{\sum_{j \in cl_f} 1/den_j}$.
- (b) Generate $[w_{intra,i} \cdot N_{intra}]$ artificial minority samples applying SMOTE in the filtered cluster, where $[]$ represents the floor function.
10. Generate N_{inter} intercluster synthetic samples for each combination of filtered clusters $i, j \in cl_f$:
 - (a) Calculate the intercluster weights as $w_{inter,ij} = \frac{1/den_{ij}}{\sum_{k,l \in cl_f} 1/den_{kl}}$.
 - (b) Generate $[w_{inter,ij} \cdot N_{inter}]$ artificial minority samples applying SMOTE between the instances of the filtered clusters i and j .

Output

The set of generated synthetic examples.

4.3. Details and justification of the SOMO algorithm

As explained above, not all the minority class instances are appropriate for generating the synthetic instances. First, they might belong to areas where the minority class examples are dominant and therefore easy for learning. Second, the opposite case might also be true i.e. they belong to areas where the majority class examples dominate. Finally, the manifold structure of the input space, which reflects the underlying data distribution, should be taken into account when synthetic minority samples are generated.

The various steps of the SOMO algorithm can be described in detail as follows:

1, 2, 3. A SOM with $SOM_{parameters}$ is applied to the input space S . The dimension of the SOM grid, N_{Grid} , is a parameter of the SOMO algorithm and its optimal value can be experimentally approximated. Nevertheless the projection of the high dimensional input space into the two dimensional SOM manifold allows us to restrict the values of N_{Grid} based on the following arguments: The application of SOM will result into a number of N_{Grid}^2 clusters which in turn will be used to identify safe areas for the data generation process. The choice of the minimum value of N_{Grid} should allow to discriminate between sparse and dense minority class areas. A very small value of this parameter will not be able to identify these sub-clusters, since the resulting clusters will have a very large size, and therefore deal with the within-class imbalance problem. A lower bound for the parameter can be calculated if we assume a uniform distribution of the minority samples on the two dimensional manifold. Then choosing this minimum dimension as $\sqrt{|S_{min}|}$ ensures that each SOM cluster contains on average one minority class sample per map unit. On the other hand any other distribution will result in sparse and dense clusters of minority class samples. An example is shown in Fig. 3(a) and (b) for a SOM of hexagonal grid topology. Regarding the maximum value of N_{Grid} , similar arguments apply. Higher values N_{Grid} will result in higher number of clusters with smaller sizes. Then the change of generating more diverse instances, i.e. generate intercluster synthetic instances, decreases since the distance between adjacent clusters decreases too. Therefore the choice $\sqrt{|S_{maj}|}$ provides a reasonable upper bound for N_{Grid} .

4, 5, 6. For each SOM cluster an imbalance ratio is defined as $IR_i = \frac{n_{-,i}+1}{n_{+,i}+1}$. The clusters that have more positive than negative examples are identified as filtered clusters. For each filtered cluster, the average Euclidean distance across all pairs of minority class instances is calculated. Fig. 4(a) presents a case where the input space includes filtered and non-filtered clusters. Also notice that, for a fixed value of N_{Grid} , in the case where no filtered clusters are found, SOMO does not generate any artificial data. The interpretation for this case is that the minority samples are sparse and the distinction between informative and noisy samples is not clear enough for SOMO to safely generate data points without introducing noise. Then the solution is to increase the value of N_{Grid} and repeat the identification of filtered clusters.

7, 8. A density factor is assigned to each filtered cluster as $den_i = \frac{n_{+,i}}{dist_i^2}$. This equation represents the (surface) density of the minority class instances on the two-dimensional map unit after applying the SOM projection on the initial input space. Also for each topologically neighboring pair of filtered clusters a total density factor is defined between them as the sum of each individual density factor. Notice that filtered clusters, which are not topologically neighbors, are excluded from the intercluster data generation process. Finally intracluster and intercluster weights are calculated based on the density factors.

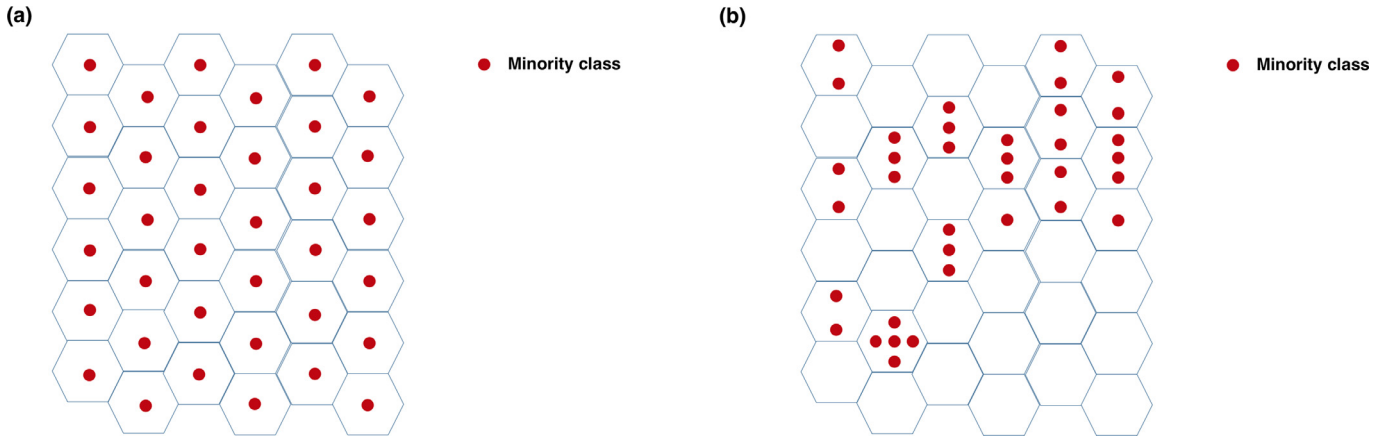


Fig. 3. (a) An example of 36 instances distributed uniformly distributed on the two-dimensional manifold of grid dimension equal to 6. Majority samples are not shown for clarity. (b) The same example assuming a non-uniform distribution. Areas dense in minority samples will be used to generate artificial data.

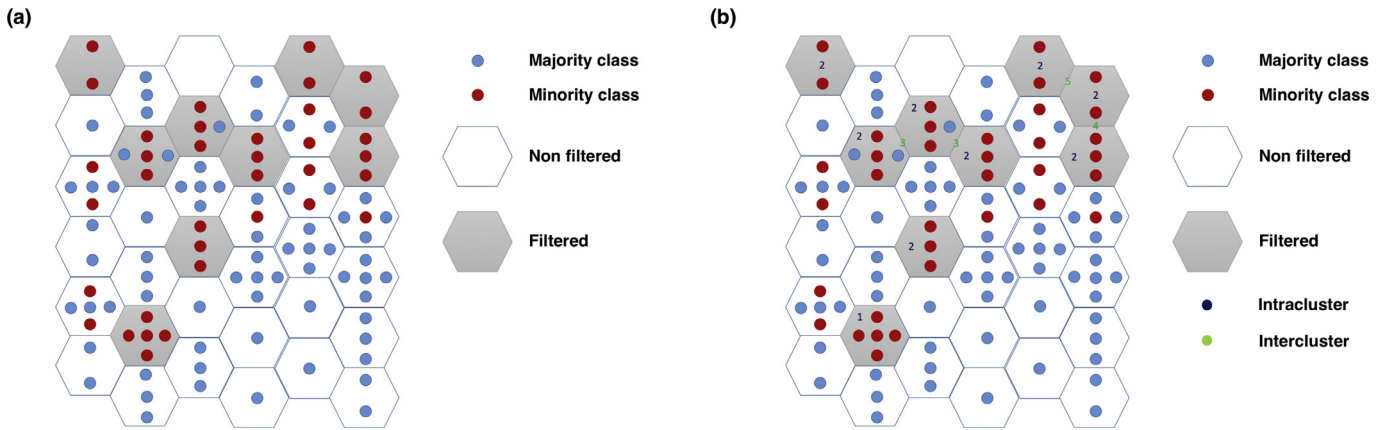


Fig. 4. An example of an imbalanced data set with 36 positive and 68 negative class samples. A SOM is applied to the data set with the proposed minimum grid dimension, equal to 6. (a) 9 filtered clusters are identified. (b) Assuming that the average Euclidean distance between pairs of positive class instances is the same for any cluster the weights are calculated and the number of intracluster and intercluster generated samples is shown.

9, 10. Filtered clusters are areas of the input space where the minority class dominates over the majority class. Therefore they can be considered as safe areas for the generation of minority samples since we expect that noisy minority examples should belong to sparse areas of minority instances i.e. non-filtered clusters. Each filtered cluster or pair of neighboring filtered clusters is assigned a weight, which is inversely proportional to their density factor, so that for relatively denser/sparser areas of positive class examples, less/more artificial examples are generated respectively. This strategy aims to deal with both the between-class imbalance and within-class imbalance problems. Fig. 4(b) presents an example of this process. The unique characteristic of SOMO is that intercluster instances are generated only between clusters, which are topologically neighbors. Therefore SOMO avoids generating data between any two clusters which are neighbors based on their Euclidean distance and may not respect the manifold structure of the input space. Fig. 5(a) and (b) present a one dimensional simplified example of this scenario where the manifold structure of the input space corresponds to a curve. Notice that clusters A and B which can be considered as neighbors in the input space based on the Euclidean distance, do not correspond to topologically neighbors and SOMO will not generate any intercluster artificial data between them, despite the fact that they are filtered clusters.

5. Research methodology

5.1. Data

In order to test the performance of SOMO we used 13 imbalanced data sets from the Machine Learning Repository UCI which have diverse characteristics. We also included 13 additional data sets that resulted from undersampling the minority class of the initial data sets such that their final IR is increased approximately by a multiplication factor of 5. Table 1 shows a summary of the 26 data sets:

5.2. Comparison of oversampling methods

The performance of SOMO was evaluated and compared against the following oversampling methods: Random Oversampling, SMOTE, Borderline SMOTE, Cluster SMOTE and ADASYN. Random Oversampling will serve as the baseline method, also being a popular approach among industry practitioners, because of its simplicity and wide implementation in commercial software. SMOTE can be considered as the most popular oversampling method, which can be confirmed by the large number of citations of the original paper describing the algorithm. Finally, Borderline SMOTE and ADASYN are two of the most important variations of SMOTE,

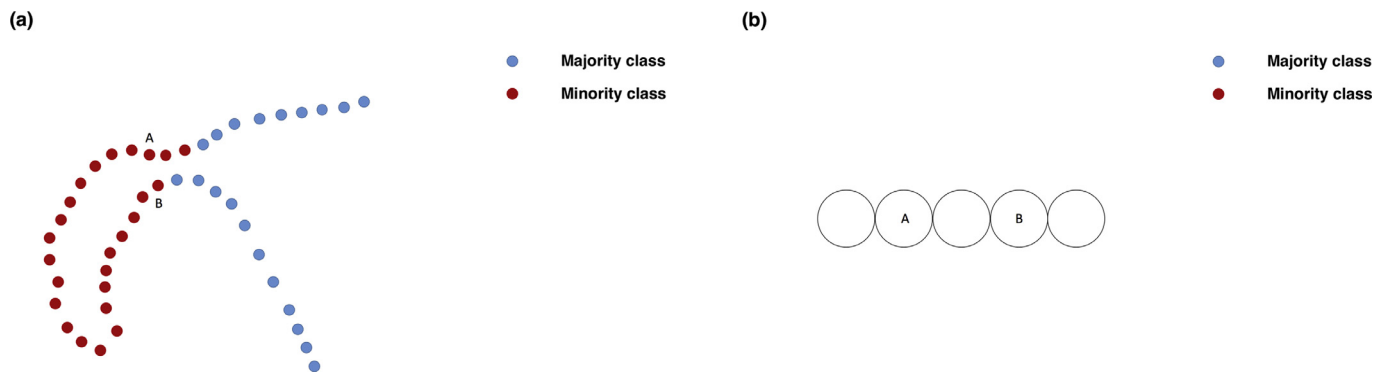


Fig. 5. (a) An example of a two-dimensional input space with an one dimensional manifold structure is shown. Two areas A and B are neighbors based on their Euclidean distance. (b) The areas A and B are included in two non-neighboring clusters of the one dimensional SOM representation of this input space.

Table 1
Description of the datasets.

Dataset	Minority class	# Features	# Instances	# Minority instances	# Majority instances	Imbalanced ratio
<i>Breast</i>	car, fad	9	106	36	70	1.94
<i>Ecoli</i>	pp	7	336	52	284	5.46
<i>Eucalyptus</i>	5	8	642	98	544	5.55
<i>Glass</i>	1	9	214	70	144	2.06
<i>Haberman</i>	2	3	306	81	225	2.78
<i>Heart</i>	1	13	270	120	150	1.25
<i>Iris</i>	2	4	150	50	100	2.00
<i>Libra</i>	1, 2, 3	90	360	72	288	4.00
<i>Liver</i>	1	6	345	145	200	1.38
<i>Pima</i>	1	8	768	268	500	1.87
<i>Segment</i>	WINDOW	16	2310	330	1980	6.00
<i>Vehicle</i>	van	18	846	199	647	3.25
<i>Wine</i>	2	13	178	71	107	1.51
Undersampled dataset						
<i>Breast</i>	car, fad	9	77	7	70	10.0
<i>Ecoli</i>	pp	7	294	10	284	28.4
<i>Eucalyptus</i>	5	8	563	19	544	28.63
<i>Glass</i>	1	9	158	14	144	10.29
<i>Haberman</i>	2	3	241	16	225	14.06
<i>Heart</i>	1	13	174	24	150	6.25
<i>Iris</i>	2	4	110	10	100	10.00
<i>Libra</i>	1, 2, 3	90	302	14	288	20.57
<i>Liver</i>	1	6	228	28	200	7.14
<i>Pima</i>	1	8	553	53	500	9.43
<i>Segment</i>	WINDOW	16	2046	66	1980	30.00
<i>Vehicle</i>	van	18	686	39	647	16.59
<i>Wine</i>	2	13	121	14	107	7.64

while Cluster SMOTE is a direct competitor of SOMO as a clustering based variation.

5.3. Performance evaluation

For the analysis of the results, an appropriate evaluation metric should be used. There are many ways to assess the quality of a classifier for binary problems. Typically, the first step is to construct a confusion matrix where each row of the matrix represents a predicted class and each column represents an observed class, as is presented in Table 2:

In this confusion matrix, TP (True Positives) is the number of positive observations (minority class) correctly classified as pos-

itive, FP (False Positives) is the number of negative observations (majority class) incorrectly classified as positive, FN (False Negatives) is the number of positive observations incorrectly classified as negative and TN is the number of negative observations correctly classified as negative. Generally, Total Accuracy could be used to evaluate the performance of a classifier and is defined as

$$\text{Total Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

For a balanced dataset, where the two classes are equally represented, Total Accuracy is a popular choice. As was explained above, for imbalanced data sets the misclassification cost of various classes might be different. This fact makes the use of predictive accuracy inappropriate or even misleading. For example, a trivial classifier that always predicts the negative class, will have a Total Accuracy equal to the proportion of negative class observations in the data set, but it will misclassify every positive class observation. In this case the Total Accuracy will be high but the accuracy for the positive class will be equal to zero.

Since prediction accuracy is not appropriate for imbalanced datasets, other performance metrics that consider the accuracy for

Table 2
The confusion matrix.

Confusion matrix	Observed positive	Observed negative
Predicted Positive	TP	FP
Predicted Negative	FN	TN

both classes should be used. In this study the following evaluation metrics are used: F-measure, G-mean and Area under the ROC Curve (AUC) (He & Garcia, 2009; Galar et al., 2012). F-measure is defined with the help of Precision and Recall. Precision is the proportion of correct positive class predictions relative to the total number of positive class predictions i.e. $Precision = \frac{TP}{TP+FP}$. Recall is the proportion of correct positive class predictions relative to the total number of positive class observations i.e. $Recall = \frac{TP}{TP+FN}$. Recall is also referred to as the True Positive Rate. Then the F-measure, assuming that Precision and Recall are equally important, is the harmonic mean of Precision and Recall, i.e.

$$F_{measure} = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

Similarly G-mean is the geometric mean of the True Positive Rate and the True Negative Rate, i.e.

$$G_{mean} = \sqrt{\frac{TP}{TP+FN} \cdot \frac{TN}{TN+FP}}$$

Finally, on the ROC curve the X-axis represents the False Positive Rate (FPR) defined as i.e. the proportion of misclassified negative examples relative to the total number of negative class observations. The Y-axis represents True Positive Rate, defined above. Varying the classification threshold of the classifier identifies different points of the ROC curve. Since the ROC curve depends on the classification threshold, the AUC (Area Under the ROC Curve) is a useful metric for the performance of the classifier as it is independent of the decision criterion.

A ranking score is applied to each method for each available data set and metric. The ranking score for the best performing method is 1 and for the worst performing method is 7, since additionally to the six oversampling algorithms we also include the performance of the classification algorithms when no oversampling is used. The Friedman test is applied to the ranking results. Generally the Friedman test is used to detect differences in the results across multiple experimental attempts, when the normality assumption may not hold. The null hypothesis is whether the classifiers have a similar performance across the oversampling methods when they are compared to their mean rankings.

5.4. Experimental procedure

For the evaluation of the oversampling methods, Logistic Regression (LR) (McCullagh, 1984) and Gradient Boosting Machine (GBM) (Friedman, 2001) were used. The intention of using two different algorithms was to draw conclusions that do not depend on any special properties of a given algorithm. Another motivation for this specific choice is the fact that LR does not have any hyperparameters to tune and the chosen GBM software implementation allows for an early stopping of the training process in order to avoid overfitting. Both of these two facts make the results less sensitive to the tuning of the parameters, and thus more reliable.

In order to evaluate the performance of the algorithms k-fold cross validation was applied with $k=5$. Before training, in each of the k stages of k-fold cross validation, synthetic data were generated for the $k-1$ folds of the data set such that the resulting training data set becomes perfectly balanced. Therefore the training data included the original data of the $k-1$ folds and the generated data. Also the tuning of GBM was done in each of the k stages using the original data of the $k-1$ folds as follows: the training data of the $k-1$ folds were split into training/validation data sets and the validation AUC was maximized. For both of the algorithms, the model predictions in the remaining fold were generated only for the original data and did not include any generated data.

Additionally, for SMOTE, Borderline SMOTE, Cluster SMOTE, ADASYN and SOMO algorithms the number of nearest neighbors

was selected among 2, 3, 4 and 5 such that the performance is maximized for each combination of datasets, metrics and classification algorithms. The same strategy was followed for the ratio of intracluster/intercluster generated points of the SOMO algorithm, resulting in values which belonged in the range between 60% and 90%. Also, following Section 4.3., the search for the optimal value of N_{Grid} started from the minimum value $\sqrt{|S_{min}|}$ and was increased up to the maximum value $\sqrt{|S_{maj}|}$ using 5 equal steps.

Finally, the experimental procedure was repeated 5 times and the reported results include the average values between the experiments as well as their standard deviation.

5.5. Software implementation

The implementation of oversampling methods was based on the Python libraries scikit-learn (Pedregosa et al., 2011) and imbalanced-learn (Lemaitre, Nogueira, & Adidas, 2016).

6. Experimental results

6.1. Comparison of the algorithms

The results for the LR classifier and for the various oversampling methods on the 26 data sets are summarized in Table 3:

The none method represents the case where no oversampling is applied. Similar results for GBM on the 26 data sets is shown in Table 4:

A ranking score is applied to each method for each available data set and metric. The ranking score for the best performing method is 1 and for the worst performing method 7. The mean ranking across the data sets for the oversampling methods and the two different algorithms is summarized in Table 5:

The results of the application of the Friedman test are shown in Table 6:

Therefore at a significance level of $\alpha=0.05$ the null hypothesis is rejected, i.e. the two classifiers do not perform similarly in mean rankings. Additionally from the mean ranking results of the previous table we conclude that SOMO performs better than the other oversampling methods. No other statistical test is required to confirm this conclusion.

6.2. SOMO parameters

SOMO requires two categories of parameters to be tuned. The first category is the SOM parameters. For the SOM parameters, which are related to the SOM training, standard strategies can be followed (Kohonen, 2001) in order to have a satisfactory convergence of the algorithm. The most important parameter of SOM that directly affects the SOMO data generation process is the grid dimension. As explained in Section 4.3., the minimum value of the grid dimension is set to $\sqrt{|S_{min}|}$ while higher values will result in the generation of less diverse synthetic instances. On the other hand the change of generating noisy samples decreases. The optimal value for this parameter depends on the characteristics of the data set. An effective strategy that was applied in our experimental procedure, is to start from the minimum value and gradually increase it up to $\sqrt{|S_{maj}|}$, by a predefined number of steps, which should be adjusted based on the available computational resources.

The second category of SOMO parameters is the number of generated intracluster and intercluster samples. Equivalently, we can use the total number of generated samples and the respective intracluster/intercluster ratio. The number of generated points was fixed, during the experimental procedure, while varying the ratio of the intracluster/intercluster generated points, we were able to detect a dependence from IR. Particularly, for an IR smaller than 10 the best results, on average, were obtained for a ratio value equal

Table 3

Results for the oversampling methods on the datasets classified using LR.

Dataset	Metric	None	Random	SMOTE	Borderline SMOTE	ADASYN	Cluster SMOTE	SOMO
<i>Breast</i>	AUC	0.930 ± 0.003	0.932 ± 0.008	0.956 ± 0.025	0.946 ± 0.009	0.932 ± 0.004	0.942 ± 0.019	0.944 ± 0.018
	F	0.510 ± 0.002	0.644 ± 0.018	0.654 ± 0.019	0.703 ± 0.018	0.693 ± 0.011	0.677 ± 0.017	0.661 ± 0.022
	G	0.558 ± 0.006	0.672 ± 0.027	0.674 ± 0.014	0.734 ± 0.015	0.713 ± 0.005	0.703 ± 0.010	0.690 ± 0.021
<i>Ecoli</i>	AUC	0.907 ± 0.000	0.903 ± 0.002	0.903 ± 0.001	0.912 ± 0.001	0.897 ± 0.001	0.895 ± 0.001	0.900 ± 0.002
	F	0.285 ± 0.001	0.716 ± 0.010	0.723 ± 0.007	0.615 ± 0.011	0.588 ± 0.003	0.665 ± 0.008	0.709 ± 0.009
	G	0.384 ± 0.001	0.874 ± 0.003	0.877 ± 0.002	0.791 ± 0.005	0.806 ± 0.002	0.842 ± 0.006	0.876 ± 0.007
<i>Eucalyptus</i>	AUC	0.815 ± 0.000	0.848 ± 0.001	0.843 ± 0.003	0.836 ± 0.009	0.860 ± 0.003	0.843 ± 0.004	0.851 ± 0.005
	F	0.263 ± 0.001	0.471 ± 0.007	0.467 ± 0.003	0.456 ± 0.018	0.477 ± 0.003	0.476 ± 0.006	0.483 ± 0.006
	G	0.398 ± 0.001	0.743 ± 0.008	0.733 ± 0.002	0.723 ± 0.020	0.740 ± 0.004	0.729 ± 0.004	0.737 ± 0.006
<i>Glass</i>	AUC	0.809 ± 0.002	0.811 ± 0.005	0.809 ± 0.003	0.811 ± 0.001	0.811 ± 0.001	0.809 ± 0.004	0.815 ± 0.007
	F	0.576 ± 0.002	0.644 ± 0.015	0.642 ± 0.009	0.650 ± 0.002	0.656 ± 0.002	0.645 ± 0.007	0.644 ± 0.012
	G	0.662 ± 0.000	0.664 ± 0.020	0.668 ± 0.015	0.646 ± 0.006	0.657 ± 0.002	0.670 ± 0.012	0.679 ± 0.018
<i>Haberman</i>	AUC	0.712 ± 0.001	0.722 ± 0.007	0.723 ± 0.004	0.722 ± 0.004	0.721 ± 0.002	0.713 ± 0.004	0.727 ± 0.009
	F	0.249 ± 0.001	0.501 ± 0.018	0.498 ± 0.010	0.485 ± 0.009	0.512 ± 0.005	0.502 ± 0.009	0.508 ± 0.016
	G	0.390 ± 0.000	0.639 ± 0.017	0.631 ± 0.024	0.621 ± 0.007	0.656 ± 0.004	0.639 ± 0.016	0.638 ± 0.023
<i>Heart</i>	AUC	0.894 ± 0.001	0.896 ± 0.002	0.894 ± 0.002	0.896 ± 0.003	0.895 ± 0.001	0.894 ± 0.004	0.906 ± 0.005
	F	0.823 ± 0.001	0.813 ± 0.002	0.813 ± 0.004	0.815 ± 0.004	0.819 ± 0.002	0.819 ± 0.003	0.823 ± 0.006
	G	0.842 ± 0.000	0.831 ± 0.002	0.832 ± 0.004	0.833 ± 0.004	0.837 ± 0.002	0.837 ± 0.006	0.837 ± 0.008
<i>Iris</i>	AUC	0.743 ± 0.001	0.724 ± 0.010	0.738 ± 0.007	0.751 ± 0.001	0.702 ± 0.002	0.719 ± 0.008	0.738 ± 0.009
	F	0.375 ± 0.001	0.653 ± 0.007	0.657 ± 0.009	0.655 ± 0.003	0.660 ± 0.004	0.657 ± 0.008	0.656 ± 0.011
	G	0.488 ± 0.002	0.637 ± 0.006	0.660 ± 0.024	0.694 ± 0.007	0.578 ± 0.006	0.621 ± 0.019	0.651 ± 0.018
<i>Libra</i>	AUC	0.451 ± 0.001	0.493 ± 0.009	0.503 ± 0.012	0.575 ± 0.013	0.554 ± 0.005	0.534 ± 0.009	0.514 ± 0.014
	F	0.155 ± 0.001	0.411 ± 0.009	0.407 ± 0.011	0.354 ± 0.017	0.381 ± 0.009	0.396 ± 0.014	0.414 ± 0.013
	G	0.236 ± 0.000	0.510 ± 0.009	0.518 ± 0.013	0.505 ± 0.014	0.532 ± 0.009	0.524 ± 0.014	0.521 ± 0.014
<i>Liver</i>	AUC	0.708 ± 0.001	0.706 ± 0.002	0.705 ± 0.003	0.704 ± 0.004	0.713 ± 0.001	0.713 ± 0.005	0.720 ± 0.007
	F	0.557 ± 0.001	0.611 ± 0.005	0.618 ± 0.011	0.611 ± 0.006	0.622 ± 0.007	0.628 ± 0.009	0.621 ± 0.010
	G	0.627 ± 0.001	0.652 ± 0.005	0.660 ± 0.009	0.650 ± 0.002	0.662 ± 0.005	0.654 ± 0.008	0.659 ± 0.007
<i>Pima</i>	AUC	0.829 ± 0.001	0.829 ± 0.001	0.829 ± 0.001	0.824 ± 0.002	0.829 ± 0.001	0.825 ± 0.005	0.832 ± 0.005
	F	0.618 ± 0.001	0.680 ± 0.004	0.679 ± 0.005	0.670 ± 0.004	0.679 ± 0.006	0.669 ± 0.007	0.680 ± 0.006
	G	0.692 ± 0.000	0.752 ± 0.003	0.751 ± 0.004	0.742 ± 0.003	0.750 ± 0.004	0.758 ± 0.005	0.763 ± 0.005
<i>Segment</i>	AUC	0.944 ± 0.001	0.942 ± 0.001	0.944 ± 0.001	0.932 ± 0.001	0.912 ± 0.001	0.925 ± 0.003	0.946 ± 0.003
	F	0.658 ± 0.001	0.640 ± 0.004	0.647 ± 0.001	0.635 ± 0.003	0.611 ± 0.002	0.638 ± 0.004	0.652 ± 0.005
	G	0.756 ± 0.002	0.883 ± 0.002	0.882 ± 0.001	0.882 ± 0.001	0.878 ± 0.001	0.874 ± 0.005	0.886 ± 0.006
<i>Vehicle</i>	AUC	0.993 ± 0.001	0.993 ± 0.010	0.993 ± 0.001	0.993 ± 0.001	0.992 ± 0.001	0.986 ± 0.001	0.995 ± 0.004
	F	0.943 ± 0.002	0.931 ± 0.003	0.940 ± 0.001	0.940 ± 0.002	0.916 ± 0.002	0.932 ± 0.005	0.944 ± 0.003
	G	0.964 ± 0.001	0.962 ± 0.001	0.964 ± 0.001	0.965 ± 0.002	0.960 ± 0.001	0.955 ± 0.001	0.970 ± 0.002
<i>Wine</i>	AUC	0.983 ± 0.002	0.980 ± 0.001	0.982 ± 0.001	0.980 ± 0.001	0.980 ± 0.001	0.973 ± 0.002	0.982 ± 0.002
	F	0.900 ± 0.001	0.898 ± 0.007	0.905 ± 0.003	0.897 ± 0.004	0.902 ± 0.006	0.905 ± 0.005	0.909 ± 0.008
	G	0.917 ± 0.001	0.916 ± 0.006	0.922 ± 0.003	0.915 ± 0.004	0.920 ± 0.004	0.914 ± 0.003	0.926 ± 0.005
Undersampled dataset								
<i>Breast</i>	AUC	0.759 ± 0.012	0.837 ± 0.023	0.834 ± 0.023	0.817 ± 0.003	0.804 ± 0.014	0.813 ± 0.021	0.830 ± 0.024
	F	0.000 ± 0.000	0.235 ± 0.078	0.198 ± 0.025	0.214 ± 0.030	0.160 ± 0.080	0.186 ± 0.053	0.213 ± 0.053
	G	0.000 ± 0.000	0.354 ± 0.080	0.334 ± 0.043	0.325 ± 0.043	0.248 ± 0.096	0.300 ± 0.072	0.334 ± 0.067
<i>Ecoli</i>	AUC	0.929 ± 0.001	0.936 ± 0.002	0.937 ± 0.001	0.936 ± 0.001	0.939 ± 0.001	0.934 ± 0.003	0.945 ± 0.006
	F	0.000 ± 0.000	0.465 ± 0.027	0.405 ± 0.009	0.463 ± 0.006	0.479 ± 0.000	0.440 ± 0.006	0.438 ± 0.018
	G	0.000 ± 0.000	0.909 ± 0.023	0.863 ± 0.004	0.916 ± 0.002	0.899 ± 0.000	0.872 ± 0.003	0.891 ± 0.011
<i>Eucalyptus</i>	AUC	0.689 ± 0.001	0.862 ± 0.003	0.858 ± 0.002	0.811 ± 0.003	0.873 ± 0.002	0.857 ± 0.006	0.863 ± 0.005
	F	0.000 ± 0.000	0.196 ± 0.004	0.216 ± 0.003	0.196 ± 0.009	0.220 ± 0.004	0.226 ± 0.005	0.213 ± 0.005
	G	0.000 ± 0.000	0.727 ± 0.002	0.737 ± 0.001	0.719 ± 0.013	0.741 ± 0.001	0.742 ± 0.002	0.739 ± 0.004
<i>Glass</i>	AUC	0.789 ± 0.001	0.783 ± 0.006	0.790 ± 0.006	0.793 ± 0.009	0.781 ± 0.005	0.779 ± 0.009	0.791 ± 0.008
	F	0.000 ± 0.000	0.370 ± 0.047	0.412 ± 0.012	0.361 ± 0.041	0.420 ± 0.013	0.424 ± 0.013	0.412 ± 0.027
	G	0.000 ± 0.000	0.596 ± 0.046	0.645 ± 0.014	0.591 ± 0.042	0.634 ± 0.014	0.648 ± 0.017	0.635 ± 0.028
<i>Haberman</i>	AUC	0.440 ± 0.001	0.541 ± 0.028	0.554 ± 0.023	0.570 ± 0.022	0.541 ± 0.004	0.556 ± 0.018	0.559 ± 0.027
	F	0.000 ± 0.000	0.152 ± 0.015	0.145 ± 0.013	0.186 ± 0.007	0.152 ± 0.002	0.150 ± 0.011	0.160 ± 0.016
	G	0.000 ± 0.000	0.367 ± 0.042	0.362 ± 0.046	0.404 ± 0.006	0.390 ± 0.003	0.380 ± 0.026	0.381 ± 0.039

Table 3 (continued)

Dataset	Metric	None	Random	SMOTE	Borderline SMOTE	ADASYN	Cluster SMOTE	SOMO
Heart	AUC	0.798 ± 0.001	0.795 ± 0.010	0.788 ± 0.006	0.797 ± 0.006	0.787 ± 0.002	0.792 ± 0.005	0.793 ± 0.010
	F	0.434 ± 0.001	0.428 ± 0.005	0.412 ± 0.013	0.426 ± 0.017	0.421 ± 0.009	0.413 ± 0.012	0.422 ± 0.013
	G	0.576 ± 0.002	0.696 ± 0.003	0.678 ± 0.013	0.682 ± 0.020	0.664 ± 0.014	0.670 ± 0.016	0.688 ± 0.013
Iris	AUC	0.855 ± 0.001	0.880 ± 0.003	0.881 ± 0.002	0.881 ± 0.002	0.832 ± 0.002	0.863 ± 0.005	0.882 ± 0.004
	F	0.000 ± 0.000	0.556 ± 0.036	0.588 ± 0.000	0.519 ± 0.024	0.474 ± 0.002	0.532 ± 0.005	0.561 ± 0.016
	G	0.000 ± 0.000	0.821 ± 0.067	0.868 ± 0.000	0.744 ± 0.060	0.742 ± 0.003	0.799 ± 0.003	0.832 ± 0.026
Libra	AUC	0.706 ± 0.002	0.695 ± 0.002	0.741 ± 0.008	0.792 ± 0.002	0.693 ± 0.002	0.727 ± 0.005	0.730 ± 0.008
	F	0.000 ± 0.000	0.258 ± 0.010	0.290 ± 0.014	0.415 ± 0.008	0.271 ± 0.009	0.276 ± 0.015	0.276 ± 0.016
	G	0.000 ± 0.000	0.637 ± 0.005	0.687 ± 0.007	0.753 ± 0.003	0.670 ± 0.015	0.683 ± 0.012	0.670 ± 0.011
Liver	AUC	0.682 ± 0.001	0.682 ± 0.013	0.692 ± 0.009	0.569 ± 0.011	0.698 ± 0.006	0.694 ± 0.011	0.692 ± 0.013
	F	0.214 ± 0.002	0.331 ± 0.013	0.328 ± 0.008	0.227 ± 0.022	0.338 ± 0.013	0.329 ± 0.014	0.334 ± 0.014
	G	0.330 ± 0.001	0.636 ± 0.016	0.647 ± 0.007	0.518 ± 0.042	0.645 ± 0.012	0.654 ± 0.013	0.649 ± 0.015
Pima	AUC	0.784 ± 0.001	0.806 ± 0.006	0.810 ± 0.004	0.818 ± 0.004	0.801 ± 0.003	0.814 ± 0.007	0.821 ± 0.009
	F	0.162 ± 0.002	0.362 ± 0.011	0.369 ± 0.011	0.365 ± 0.006	0.354 ± 0.005	0.371 ± 0.011	0.373 ± 0.014
	G	0.269 ± 0.001	0.718 ± 0.007	0.723 ± 0.012	0.709 ± 0.005	0.726 ± 0.002	0.721 ± 0.007	0.727 ± 0.010
Segment	AUC	0.917 ± 0.002	0.921 ± 0.001	0.921 ± 0.001	0.916 ± 0.003	0.921 ± 0.001	0.915 ± 0.002	0.927 ± 0.005
	F	0.405 ± 0.001	0.260 ± 0.004	0.270 ± 0.004	0.266 ± 0.005	0.252 ± 0.002	0.259 ± 0.007	0.273 ± 0.006
	G	0.512 ± 0.002	0.853 ± 0.003	0.851 ± 0.005	0.840 ± 0.006	0.853 ± 0.003	0.852 ± 0.007	0.860 ± 0.008
Vehicle	AUC	0.991 ± 0.002	0.992 ± 0.000	0.992 ± 0.001	0.991 ± 0.000	0.990 ± 0.000	0.984 ± 0.004	0.993 ± 0.001
	F	0.808 ± 0.001	0.835 ± 0.006	0.840 ± 0.007	0.826 ± 0.006	0.816 ± 0.017	0.835 ± 0.014	0.847 ± 0.010
	G	0.881 ± 0.007	0.937 ± 0.005	0.928 ± 0.001	0.914 ± 0.000	0.929 ± 0.011	0.923 ± 0.009	0.936 ± 0.007
Wine	AUC	0.899 ± 0.002	0.884 ± 0.001	0.883 ± 0.004	0.889 ± 0.003	0.899 ± 0.012	0.895 ± 0.009	0.897 ± 0.007
	F	0.560 ± 0.001	0.569 ± 0.014	0.580 ± 0.004	0.578 ± 0.017	0.597 ± 0.032	0.598 ± 0.020	0.592 ± 0.016
	G	0.630 ± 0.001	0.756 ± 0.013	0.753 ± 0.003	0.759 ± 0.013	0.736 ± 0.046	0.742 ± 0.028	0.753 ± 0.018

The bold values represent the best oversampling algorithm for each row.

Table 4

Results for the oversampling methods on the datasets classified using GBM.

Dataset	Metric	None	Random	SMOTE	Borderline SMOTE	ADASYN	Cluster SMOTE	SOMO
Breast	AUC	0.730 ± 0.002	0.788 ± 0.014	0.783 ± 0.041	0.782 ± 0.037	0.825 ± 0.020	0.797 ± 0.032	0.797 ± 0.030
	F	0.394 ± 0.002	0.478 ± 0.029	0.503 ± 0.041	0.527 ± 0.027	0.593 ± 0.043	0.547 ± 0.042	0.518 ± 0.041
	G	0.469 ± 0.001	0.513 ± 0.023	0.546 ± 0.051	0.568 ± 0.037	0.630 ± 0.052	0.579 ± 0.054	0.546 ± 0.043
Ecoli	AUC	0.936 ± 0.001	0.940 ± 0.005	0.931 ± 0.007	0.934 ± 0.002	0.939 ± 0.006	0.943 ± 0.010	0.947 ± 0.011
	F	0.823 ± 0.001	0.761 ± 0.015	0.762 ± 0.013	0.739 ± 0.007	0.681 ± 0.004	0.722 ± 0.009	0.754 ± 0.015
	G	0.890 ± 0.001	0.882 ± 0.007	0.881 ± 0.012	0.867 ± 0.001	0.853 ± 0.005	0.871 ± 0.010	0.881 ± 0.010
Eucalyptus	AUC	0.826 ± 0.001	0.850 ± 0.009	0.851 ± 0.003	0.846 ± 0.003	0.860 ± 0.004	0.857 ± 0.005	0.859 ± 0.007
	F	0.293 ± 0.002	0.481 ± 0.010	0.502 ± 0.014	0.501 ± 0.012	0.499 ± 0.020	0.504 ± 0.021	0.496 ± 0.018
	G	0.391 ± 0.001	0.753 ± 0.013	0.747 ± 0.011	0.748 ± 0.012	0.734 ± 0.015	0.749 ± 0.014	0.759 ± 0.014
Glass	AUC	0.820 ± 0.003	0.864 ± 0.011	0.864 ± 0.015	0.881 ± 0.013	0.853 ± 0.008	0.864 ± 0.013	0.866 ± 0.014
	F	0.675 ± 0.002	0.675 ± 0.018	0.676 ± 0.022	0.690 ± 0.015	0.679 ± 0.014	0.680 ± 0.022	0.680 ± 0.023
	G	0.747 ± 0.002	0.747 ± 0.015	0.748 ± 0.017	0.759 ± 0.013	0.755 ± 0.011	0.743 ± 0.017	0.751 ± 0.017
Haberman	AUC	0.640 ± 0.001	0.629 ± 0.008	0.630 ± 0.010	0.621 ± 0.017	0.636 ± 0.022	0.641 ± 0.018	0.635 ± 0.015
	F	0.266 ± 0.001	0.370 ± 0.026	0.351 ± 0.021	0.360 ± 0.021	0.394 ± 0.019	0.371 ± 0.022	0.375 ± 0.024
	G	0.365 ± 0.002	0.472 ± 0.036	0.454 ± 0.027	0.475 ± 0.026	0.485 ± 0.026	0.467 ± 0.030	0.466 ± 0.033
Heart	AUC	0.885 ± 0.001	0.889 ± 0.004	0.887 ± 0.003	0.888 ± 0.003	0.885 ± 0.003	0.895 ± 0.003	0.901 ± 0.005
	F	0.779 ± 0.002	0.776 ± 0.007	0.792 ± 0.011	0.796 ± 0.008	0.774 ± 0.009	0.780 ± 0.012	0.794 ± 0.011
	G	0.803 ± 0.001	0.799 ± 0.006	0.814 ± 0.010	0.816 ± 0.006	0.797 ± 0.008	0.814 ± 0.009	0.812 ± 0.010
Iris	AUC	0.982 ± 0.001	0.980 ± 0.002	0.985 ± 0.002	0.986 ± 0.001	0.953 ± 0.004	0.964 ± 0.004	0.982 ± 0.005
	F	0.873 ± 0.001	0.877 ± 0.017	0.896 ± 0.016	0.912 ± 0.001	0.907 ± 0.004	0.895 ± 0.010	0.893 ± 0.017
	G	0.905 ± 0.001	0.908 ± 0.015	0.925 ± 0.014	0.939 ± 0.001	0.933 ± 0.005	0.920 ± 0.010	0.928 ± 0.016
Libra	AUC	0.772 ± 0.004	0.765 ± 0.027	0.798 ± 0.006	0.755 ± 0.021	0.752 ± 0.022	0.766 ± 0.017	0.784 ± 0.018
	F	0.556 ± 0.012	0.530 ± 0.029	0.589 ± 0.029	0.535 ± 0.034	0.595 ± 0.026	0.598 ± 0.028	0.578 ± 0.029
	G	0.662 ± 0.012	0.660 ± 0.025	0.703 ± 0.024	0.664 ± 0.024	0.711 ± 0.013	0.710 ± 0.021	0.692 ± 0.026
Liver	AUC	0.761 ± 0.001	0.759 ± 0.005	0.762 ± 0.006	0.763 ± 0.008	0.758 ± 0.009	0.760 ± 0.008	0.762 ± 0.007
	F	0.638 ± 0.001	0.627 ± 0.008	0.623 ± 0.007	0.647 ± 0.024	0.632 ± 0.008	0.620 ± 0.011	0.625 ± 0.010
	G	0.674 ± 0.001	0.661 ± 0.006	0.659 ± 0.007	0.674 ± 0.025	0.651 ± 0.010	0.661 ± 0.009	0.663 ± 0.010

Table 4 (continued)

Dataset	Metric	None	Random	SMOTE	Borderline SMOTE	ADASYN	Cluster SMOTE	SOMO
Pima	AUC	0.834 ± 0.001	0.836 ± 0.002	0.835 ± 0.002	0.832 ± 0.001	0.826 ± 0.003	0.840 ± 0.004	0.842 ± 0.005
	F	0.635 ± 0.001	0.683 ± 0.008	0.683 ± 0.003	0.689 ± 0.002	0.672 ± 0.005	0.675 ± 0.006	0.688 ± 0.007
	G	0.710 ± 0.001	0.754 ± 0.007	0.754 ± 0.003	0.759 ± 0.001	0.746 ± 0.004	0.752 ± 0.004	0.765 ± 0.007
Segment	AUC	0.996 ± 0.001	0.996 ± 0.001	0.996 ± 0.001	0.995 ± 0.001	0.995 ± 0.001	0.997 ± 0.002	0.997 ± 0.005
	F	0.939 ± 0.002	0.918 ± 0.002	0.925 ± 0.005	0.922 ± 0.002	0.917 ± 0.005	0.915 ± 0.007	0.929 ± 0.006
	G	0.959 ± 0.001	0.961 ± 0.003	0.970 ± 0.003	0.969 ± 0.003	0.972 ± 0.002	0.969 ± 0.002	0.976 ± 0.003
Vehicle	AUC	0.993 ± 0.001	0.992 ± 0.001	0.992 ± 0.001	0.992 ± 0.001	0.992 ± 0.001	0.996 ± 0.001	0.996 ± 0.002
	F	0.919 ± 0.001	0.920 ± 0.004	0.914 ± 0.006	0.917 ± 0.002	0.915 ± 0.003	0.921 ± 0.005	0.927 ± 0.005
	G	0.947 ± 0.001	0.964 ± 0.002	0.955 ± 0.004	0.956 ± 0.001	0.961 ± 0.002	0.965 ± 0.004	0.970 ± 0.005
Wine	AUC	0.979 ± 0.001	0.982 ± 0.001	0.988 ± 0.003	0.987 ± 0.002	0.985 ± 0.001	0.989 ± 0.003	0.991 ± 0.003
	F	0.864 ± 0.005	0.872 ± 0.010	0.856 ± 0.009	0.859 ± 0.008	0.888 ± 0.020	0.870 ± 0.017	0.867 ± 0.015
	G	0.885 ± 0.004	0.893 ± 0.008	0.878 ± 0.008	0.880 ± 0.008	0.905 ± 0.017	0.896 ± 0.013	0.898 ± 0.011
Undersampled dataset								
Breast	AUC	0.469 ± 0.005	0.394 ± 0.055	0.508 ± 0.095	0.379 ± 0.138	0.649 ± 0.082	0.575 ± 0.089	0.493 ± 0.082
	F	0.080 ± 0.001	0.136 ± 0.081	0.248 ± 0.080	0.106 ± 0.012	0.145 ± 0.053	0.200 ± 0.068	0.206 ± 0.079
	G	0.131 ± 0.001	0.240 ± 0.081	0.342 ± 0.080	0.219 ± 0.020	0.279 ± 0.077	0.314 ± 0.080	0.304 ± 0.082
Ecoli	AUC	0.889 ± 0.002	0.816 ± 0.019	0.845 ± 0.008	0.848 ± 0.006	0.864 ± 0.012	0.847 ± 0.010	0.843 ± 0.013
	F	0.020 ± 0.040	0.424 ± 0.019	0.424 ± 0.043	0.494 ± 0.036	0.382 ± 0.020	0.397 ± 0.034	0.421 ± 0.035
	G	0.028 ± 0.056	0.669 ± 0.001	0.640 ± 0.054	0.693 ± 0.027	0.772 ± 0.019	0.703 ± 0.038	0.683 ± 0.032
Eucalyptus	AUC	0.883 ± 0.001	0.863 ± 0.005	0.879 ± 0.002	0.875 ± 0.006	0.887 ± 0.005	0.888 ± 0.005	0.881 ± 0.006
	F	0.471 ± 0.007	0.310 ± 0.005	0.399 ± 0.009	0.381 ± 0.019	0.393 ± 0.012	0.390 ± 0.012	0.374 ± 0.011
	G	0.626 ± 0.001	0.713 ± 0.002	0.726 ± 0.002	0.610 ± 0.033	0.705 ± 0.018	0.712 ± 0.010	0.720 ± 0.006
Glass	AUC	0.717 ± 0.002	0.748 ± 0.042	0.767 ± 0.022	0.794 ± 0.010	0.833 ± 0.030	0.796 ± 0.028	0.782 ± 0.034
	F	0.000 ± 0.000	0.376 ± 0.044	0.441 ± 0.057	0.401 ± 0.048	0.304 ± 0.071	0.381 ± 0.065	0.405 ± 0.059
	G	0.000 ± 0.000	0.622 ± 0.069	0.690 ± 0.078	0.660 ± 0.081	0.518 ± 0.088	0.604 ± 0.084	0.644 ± 0.077
Haberman	AUC	0.528 ± 0.005	0.500 ± 0.017	0.458 ± 0.026	0.569 ± 0.017	0.389 ± 0.025	0.427 ± 0.026	0.471 ± 0.025
	F	0.010 ± 0.001	0.112 ± 0.023	0.122 ± 0.012	0.218 ± 0.009	0.106 ± 0.004	0.118 ± 0.010	0.128 ± 0.015
	G	0.046 ± 0.001	0.366 ± 0.049	0.367 ± 0.048	0.421 ± 0.012	0.333 ± 0.005	0.352 ± 0.028	0.369 ± 0.043
Heart	AUC	0.831 ± 0.006	0.812 ± 0.006	0.819 ± 0.009	0.811 ± 0.006	0.804 ± 0.006	0.805 ± 0.008	0.813 ± 0.010
	F	0.223 ± 0.025	0.464 ± 0.016	0.493 ± 0.033	0.471 ± 0.028	0.357 ± 0.051	0.427 ± 0.046	0.465 ± 0.034
	G	0.313 ± 0.018	0.645 ± 0.016	0.659 ± 0.028	0.638 ± 0.015	0.518 ± 0.059	0.597 ± 0.046	0.640 ± 0.033
Iris	AUC	0.912 ± 0.001	0.890 ± 0.019	0.954 ± 0.008	0.958 ± 0.009	0.931 ± 0.006	0.951 ± 0.009	0.934 ± 0.015
	F	0.767 ± 0.001	0.744 ± 0.039	0.691 ± 0.062	0.733 ± 0.049	0.736 ± 0.003	0.723 ± 0.034	0.724 ± 0.049
	G	0.869 ± 0.002	0.874 ± 0.024	0.845 ± 0.031	0.835 ± 0.031	0.907 ± 0.001	0.883 ± 0.018	0.875 ± 0.027
Libra	AUC	0.547 ± 0.027	0.567 ± 0.059	0.775 ± 0.024	0.729 ± 0.031	0.635 ± 0.050	0.699 ± 0.039	0.692 ± 0.041
	F	0.100 ± 0.002	0.078 ± 0.030	0.258 ± 0.081	0.177 ± 0.023	0.073 ± 0.019	0.173 ± 0.054	0.178 ± 0.056
	G	0.115 ± 0.001	0.187 ± 0.054	0.539 ± 0.104	0.470 ± 0.037	0.253 ± 0.006	0.393 ± 0.057	0.373 ± 0.074
Liver	AUC	0.543 ± 0.003	0.584 ± 0.028	0.530 ± 0.029	0.521 ± 0.019	0.595 ± 0.018	0.565 ± 0.024	0.565 ± 0.030
	F	0.057 ± 0.001	0.218 ± 0.022	0.210 ± 0.026	0.193 ± 0.007	0.303 ± 0.033	0.255 ± 0.033	0.235 ± 0.029
	G	0.082 ± 0.001	0.442 ± 0.038	0.439 ± 0.040	0.439 ± 0.026	0.561 ± 0.031	0.509 ± 0.036	0.465 ± 0.041
Pima	AUC	0.810 ± 0.001	0.812 ± 0.006	0.807 ± 0.004	0.805 ± 0.005	0.816 ± 0.006	0.811 ± 0.006	0.810 ± 0.009
	F	0.370 ± 0.002	0.401 ± 0.014	0.399 ± 0.008	0.392 ± 0.009	0.403 ± 0.008	0.405 ± 0.010	0.413 ± 0.012
	G	0.522 ± 0.002	0.717 ± 0.013	0.716 ± 0.016	0.694 ± 0.013	0.711 ± 0.014	0.714 ± 0.019	0.722 ± 0.017
Segment	AUC	0.982 ± 0.003	0.984 ± 0.001	0.990 ± 0.001	0.989 ± 0.002	0.982 ± 0.001	0.986 ± 0.004	0.997 ± 0.006
	F	0.701 ± 0.013	0.769 ± 0.016	0.753 ± 0.017	0.748 ± 0.009	0.758 ± 0.012	0.759 ± 0.018	0.770 ± 0.020
	G	0.743 ± 0.009	0.852 ± 0.006	0.896 ± 0.010	0.880 ± 0.009	0.877 ± 0.011	0.880 ± 0.014	0.879 ± 0.013
Vehicle	AUC	0.965 ± 0.001	0.965 ± 0.004	0.971 ± 0.006	0.970 ± 0.006	0.976 ± 0.005	0.968 ± 0.008	0.976 ± 0.007
	F	0.636 ± 0.013	0.654 ± 0.030	0.657 ± 0.036	0.687 ± 0.029	0.688 ± 0.026	0.677 ± 0.034	0.674 ± 0.036
	G	0.730 ± 0.008	0.796 ± 0.017	0.794 ± 0.025	0.804 ± 0.014	0.840 ± 0.020	0.809 ± 0.025	0.801 ± 0.025
Wine	AUC	0.872 ± 0.001	0.826 ± 0.045	0.802 ± 0.020	0.775 ± 0.025	0.876 ± 0.012	0.830 ± 0.018	0.822 ± 0.028
	F	0.781 ± 0.002	0.596 ± 0.044	0.582 ± 0.040	0.563 ± 0.043	0.717 ± 0.037	0.652 ± 0.041	0.612 ± 0.046
	G	0.863 ± 0.001	0.738 ± 0.050	0.753 ± 0.022	0.696 ± 0.060	0.844 ± 0.013	0.804 ± 0.020	0.773 ± 0.033

The bold values represent the best oversampling algorithm for each row.

Table 5

Results for mean ranking of the oversampling methods averaged over the datasets.

Metric	None	Random	SMOTE	Borderline SMOTE	ADASYN	Cluster SMOTE	SOMO
<i>Algorithm: LR</i>							
AUC	4.85	4.00	3.62	3.65	4.65	5.15	2.08
F	5.81	4.19	3.77	4.35	3.81	3.54	2.54
G	6.46	3.69	3.46	4.35	3.73	3.81	2.50
<i>Algorithm: GBM</i>							
AUC	4.62	4.73	3.92	4.50	4.19	3.23	2.81
F	5.19	4.62	3.85	3.62	3.92	3.65	3.15
G	6.00	4.39	3.81	3.77	3.65	3.46	2.92

Table 6

Results for Friedman's test.

Metric	p-value
<i>Algorithm: LR</i>	
AUC	6.12E-16
F	4.46E-08
G	2.25E-11
<i>Algorithm: GBM</i>	
AUC	4.31E-05
F	2.04E-03
G	1.75E-06

to 0.70. On the other hand for an IR higher than 10 the corresponding intracluster/intercluster optimal ratio was 0.85. Generally the experiments show a monotonic increasing relation between the optimal intracluster/intercluster ratio and IR. This effect can be explained by observing that lower values of this parameter will generate points that expand the minority class area in the input space. This choice of ratio values may be effective when SOM is able to capture the manifold structure of the input space, a task that becomes more difficult as IR increases since there is less information about the minority class available. Therefore for higher values of IR the optimal strategy is to also choose higher values of the intracluster/intercluster ratio. This will result in a behavior similar to SMOTE, although the generation of synthetic samples will happen in safe areas of the input space, so that noisy samples generation is avoided.

7. Conclusions

In this paper we presented SOMO, a new oversampling algorithm. SOMO generates synthetic observations by exploring the manifold structure of the data, through the topology preserving property of the SOM algorithm. A SOM is applied to the input space, data points are partitioned into clusters and synthetic data points for the minority class are generated, using instances of the minority class that belong to the same or adjacent clusters. SOMO performance was evaluated on 26 datasets with different imbalance ratios and compared to multiple oversampling methods, using Logistic Regression and Gradient Boosting Machine as classifiers. The results show that SOMO performs better compared to the other methods.

The explanation for this improvement in performance relates to the ability of SOMO to extract an approximate two-dimensional representation of the topological features of the high-dimensional space, as well as estimate the density of majority and minority instances in different areas of the space and therefore safely generate artificial data. SOMO can be a useful tool for researchers and practitioners since it results in the generation of high quality artificial data and only requires the tuning of a small number of parameters.

References

- Aggarwal, C. C., Hinneburg, A., & Keim, D. A. (2001). On the surprising behavior of distance metrics in high dimensional space. *Database Theory – ICDT, 2001*, 420–434.
- Akbani, R., Kwek, S., & Japkowicz, N. (2004). Applying support vector machine to imbalanced datasets. *Machine Learning: ECML, 2004*, 39–50.
- Barua, S., Islam, Md. M., Yao, X., & Murase, K. (2014). MWMOTE – Majority weighted minority oversampling technique for imbalanced data set learning. *IEEE Transactions on Knowledge and Data Engineering*, 26(2), 405–425.
- Batista, G. E., Prati, R. C., & Monard, M. C. (2004). A study of the behavior of several methods for balancing machine learning training data. *ACM SIGKDD Explorations Newsletter – Special Issue on Learning from Imbalanced Datasets*, 6(1), 20–29.
- Bellman, R. E. (1961). In *Adaptive control processes: A guided tour*: 28 (pp. 1–19). Princeton University Press. <http://arxiv.org/abs/1302.6677>.
- Beyer, K. S., Goldstein, J., Ramakrishnan, R., & Shaft, U. (1999). When is 'nearest Neighbor' meaningful? In *Database theory–ICDT'99* (pp. 217–235).
- Bunkhumpornpat, C., Sinapiromsaran, K., & Lursinsap, C. (2009). Safe-Level-SMOTE: Safe-level-synthetic minority over-sampling technique for handling the class imbalanced problem. In *Lecture notes in computer science (including sub-series lecture notes in artificial intelligence and lecture notes in bioinformatics)* (pp. 475–482).
- Bunkhumpornpat, C., Sinapiromsaran, K., & Lursinsap, C. (2012). DBSMOTE: Density-based synthetic minority over-sampling technique. *Applied Intelligence*, 36(3), 664–684.
- Chawla, N. V. (2010). Data mining for imbalanced datasets: An overview. In *Data Mining and Knowledge Discovery Handbook* (pp. 875–886).
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- Chawla, N. V., Japkowicz, N., & Drive, P. (2004). Editorial: Special issue on learning from imbalanced data sets. *ACM SIGKDD Explorations Newsletter*, 6(1), 1–6.
- Chawla, N. V., Japkowicz, N., & Kolcz, A. (2003a). Workshop learning from imbalanced data sets II. In *Proceedings of the international conference machine learning*.
- Chawla, N. V., Lazarevic, A., Hall, L. O., & Bowyer, K. W. (2003b). SMOTEBoost: Improving prediction of the minority class in boosting. In *Proceedings of the seventh European conference on principles and practice of knowledge discovery in databases* (pp. 107–119).
- Cieslak, D. A., & Chawla, N. V. (2008). Start globally, optimize locally, predict globally: Improving performance on imbalanced data. In *Proceedings – IEEE international conference on data mining, ICDM* (pp. 143–152).
- Cieslak, D. A., Chawla, N. V., & Striegel, A. (2006). Combating imbalance in network intrusion datasets. In *IEEE International conference on granular computing* (pp. 732–737).
- Clearwater, S. H., & Stern, E. G. (1991). A rule-learning program in high energy physics event classification. *Computer Physics Communications*, 67(2), 159–182.
- Cottrell, M., Fort, J. C., & Pagès, G. (1998). Theoretical aspects of the SOM algorithm. *Neurocomputing*, 21(1–3), 119–138.
- Domingos, P. (1999). MetaCost: A general method for making classifiers. In *Proceedings of the 5th international conference on knowledge discovery and data mining* (pp. 155–164).
- Fernández, A., López, V., Galar, M., Jesus, M. J., & Herrera, F. (2013). Analysing the classification of imbalanced data-sets with multiple classes: Binarization techniques and ad-hoc approaches. *Knowledge-Based Systems*, 42, 97–110.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
- Galar, M., Fernández, A., Barrenechea, E., Bustince, H., & Herrera, F. (2012). A review on ensembles for the class imbalance problem: Bagging-, Boosting-, and Hybrid-based approaches. *IEEE Transactions on Systems, Man and Cybernetics Part C*, 42(4), 463–484.
- Gorban, A., Kegl, B., Wunsch, D., & Zinovyen, A. (2008). *Principal manifolds for data visualization and dimension reduction*. Springer.
- Graves, S. J., Asner, G. P., Martin, R. E., Anderson, C. B., Colgan, M. S., Kalantari, L., et al. (2016). Tree species abundance predictions in a tropical agricultural landscape with a supervised classification model and imbalanced data. *Remote Sensing*, 8(2).
- Guo, H., & Viktor, H. (2004). Learning from imbalanced data sets with boosting and data generation: The DataBoost IM Approach. *ACM SIGKDD Explorations Newsletter*, 6(1), 30–39.

- Han, H., Wang, W., & Mao, B. (2005). Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning. *Advances in intelligent computing*, 17(12), 878–887.
- Hart, P. (1968). The condensed nearest neighbor rule. *IEEE Trans. Inform. Theory*, 14, 515–516.
- He, H., Bai, Y., Garcia, E., & Li, S. (2008). ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *IEEE international joint conference on neural networks*, 2008. IJCNN 2008. (IEEE world congress on computational intelligence) (pp. 1322–1328). (pp. 1322–1328).
- He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284.
- Japkowicz, N. (2000). Learning from imbalanced data sets. In *Proceedings of the american association for artificial intelligence (AAAI) workshop*.
- Jo, T., & Japkowicz, N. (2004). Class imbalances versus small disjuncts. *ACM Sigkdd Explorations Newsletter*, 6, 40–49.
- Kaski, S., & Kohonen, T. (1998). Methods for interpreting a self-organized map in data analysis. In *ESANN '98, 6th European symposium on artificial neural neural networks*, 22–24 April 1998 (pp. 185–190).
- Kaski, S., & Lagus, K. (1996). Comparing self-organizing maps. In *Lecture notes in computer science (including subseries lecture notes in artificial intelligence and lecture notes in bioinformatics)* (pp. 809–814).
- Kohonen, T. (1982). Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43(1), 59–69.
- Kohonen, T. (2001). Self-organizing maps. In *Springer series in information sciences* (p. 501).
- Kotsiantis, S., Kanellopoulos, D., & Panayiotis, P. (2006). Handling imbalanced datasets: A review. *Science*, 30(1), 25–36.
- Kubat, M., & Matwin, S. (1997). Addressing the curse of imbalanced training sets: One sided selection. *ICML*, 97, 179–186.
- Laurikkala, J. (2001). Improving identification of difficult small classes by balancing class distribution. *Artificial Intelligence in Medicine*, 63–66.
- Lemaitre, G., Nogueira, F., & Adidas, C. (2016). "Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning". CoRR abs/1609.06570.
- McCullagh, P. (1984). Generalized linear models. *European Journal of Operational Research*, 16, 285–292.
- Mefenyi, E., Jain, A., & Villmann, T. (2007). Explicit magnification control of self-organizing maps for 'forbidden' data. *IEEE Transactions on Neural Networks*, 18(3), 786–797.
- Merenyi, E., Tasdemir, K., & Zhang, L. (2009). Learning highly structured manifolds: Harnessing the power of SOMs. Springer. *Similarity-Based Clustering*, LNAI, 5400, 138–168.
- Nekooimehr, I., & Lai-Yuen, S. K. (2016). Adaptive semi-supervised weighted oversampling (A-SUWO) for imbalanced datasets. *Expert Systems with Applications*, 46, 405–416.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Prati, R. C., Batista, G. E., & Silva, D. F. (2014). Class imbalance revisited: A new experimental setup to assess the performance of treatment methods. *Knowledge and Information Systems*, 1–24.
- Ritter, H., & Schulten, K. (1988). Extending Kohonen's self-organizing mapping algorithm to learn ballistic movements. In *Neural Computers* (pp. 393–406).
- Sarajedini, A., & Chau, P. M. (1996). Cumulative distribution estimation with neural networks. In *WCNN'96* (pp. 876–880).
- Sun, Z., Song, Q., Zhu, X., Sun, H., Xu, B., & Zhou, Y. (2015). A novel ensemble method for classifying imbalanced data. *Pattern Recognition*, 48, 1623–1637.
- Tang, B., & He, H. (2015). KernelADASYN: Kernel based adaptive synthetic data generation for imbalanced learning. *IEEE Congress on Evolutionary Computation (CEC)*.
- Ting, K. M. (2002). An instance-weighting method to induce cost-sensitive trees. *IEEE Transactions on Knowledge and Data Engineering*, 14(3), 659–665.
- Tomek, I. (1976). Two modifications of CNN. *IEEE Transactions on Systems, Man and Cybernetics*, 6, 769–772.
- Verbeke, W., Dejaeger, K., Martens, D., Hur, J., & Baesens, B. (2012). New insights into churn prediction in the telecommunication sector: A profit driven data mining approach. *European Journal of Operational Research*, 218(1), 211–229.
- Wang, S., Minku, L. L., & Yao, X. (2015). Resampling-based ensemble methods for on-line class imbalance learning. *IEEE Trans. Knowledge Data Eng.*, 27(5), 1356–1368.
- Wilson, D. L. (1972). Asymptotic properties of nearest neighbor rules using edited data. *IEEE Transactions on Systems, Man and Cybernetics*, 2(3), 408–421.
- Zhao, X. M., Li, X., Chen, L., & Aihara, K. (2008). Protein classification with imbalanced data. *Proteins: Structure, Function and Genetics*, 70(4), 1125–1132.